```
using System;

namespace OOPActivity
{
    // Line 1: Abstract class representing a generic vehicle
    public abstract class Vehicle
    {
        // Line 2: Private field (Restricted access)
        private string _serialNumber;

        // Line 3: Public Property to safely access the private field
        public string SerialNumber
        {
            get { return _serialNumber; }
            set { _serialNumber = value; }
        }

        public string Model { get; set; }

        // Line 4: An abstract method with no body
        public abstract void StartEngine();
    }

    // Line 5: Car inherits from Vehicle
    public class Car : Vehicle
    {
        // Line 6: A standard method unique to Car
        public void Drive()
        {
            Console.WriteLine("The car is driving down the road.");
        }

        // Line 7: Overriding the abstract method from the parent class
        public override void StartEngine()
        {
            Console.WriteLine("The car engine roars to life softly.");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            // Line 8: Creating an instance of the Car class
            Car myCar = new Car();
```

```
            myCar.Model = "Sedan";
            myCar.SerialNumber = "XYZ123";

            // Line 9: Calling methods
            myCar.StartEngine();
            myCar.Drive();
        }
    }
}
```

Part 1: Core Building Blocks

Identify the specific line numbers or terms from the code above that represent these fundamental concepts:

1. **Class:** Give one example of a class name used in the code.
    - *Answer:* _____
2. **Object:** Which line number shows the actual creation (instantiation) of an object?
    - *Answer:* _____
3. **Property:** Name one property found inside the `Vehicle` class.
    - *Answer:* _____
4. **Method:** Name one method found inside the `Car` class.
    - *Answer:* _____

Part 2: The 4 Pillars of OOP

Match the code implementation to its correct OOP Pillar (**Encapsulation, Abstraction, Inheritance, Polymorphism**).

5. _____ : Demonstrated in **Line 3** (and Line 2), where a private field `_serialNumber` is hidden from direct outside access and safely exposed through a public `SerialNumber` property.
6. _____ : Demonstrated in **Line 4**, where `public abstract class Vehicle` and `public abstract void StartEngine();` hide the complex implementation details and only define *what* a vehicle should do, not *how*.
7. _____ : Demonstrated in **Line 5** (`public class Car : Vehicle`), where the `Car` class automatically acquires the properties and traits of the `Vehicle` class.
8. _____ : Demonstrated in **Line 7** (`public override void StartEngine()`), where the `Car` class provides its own specific implementation of a method that was originally defined in the parent class.